

PROgramming in LOGic

CS 331 Tutorial # 2

Rashmi Dutta Baruah

Department of Computer Science & Engineering

IIT Guwahati

Outline

- Lists
- Arithmetic
- Controlling backtracking: ‘cut’ and ‘fail’

Lists

- Ordered sequence of elements.
- Syntax: start and end with square brackets, and each of the elements they contain is separated by a comma.
 - Example: `[a, man, X]`
- List is either empty, having no elements, written as `[]` or is a structure that has two components: head and tail.
- Head : first element, Tail: list containing every element except the first.
- Special symbol ' | ' splits the list into head and tail.
 - Example: `[one, two, three] = [X|Y]`
 - X unifies with one i.e. `X = one` and `Y = [two, three]`

Lists

- some more examples:

```
p([Head|Tail], Head, Tail).
```

```
?- p([one,two,three], X, Y).
```

```
X = one
```

```
Y = [two, three]
```

```
yes
```

```
?- p([one], X, Y).
```

```
X = one
```

```
Y = []
```

```
yes
```

```
?- p([], X, Y).
```

```
No
```

Does (A) unifies with (B) if so then write down the instantiations?

(A) [one]

(B) [Two]

Yes, Two = one

(A) [one, two, three, four]

(B) [one, two, Y]

No

(A) [one, []]

(B) [one, X|Y]

Yes, X = [], Y = []

(A) [one, two, three]

(B) [X1, X2, X3|Y]

Yes, X1=one, X2=two, X3=three, Y=[]

Lists

- List searching
 - Lists can be used to store information and then can be subsequently searched by asking questions.
 - Example: [adi, anish, amit, anurag], students who are awarded the scholarship. Find if Anil is among the students.
 - If we see list as a set then this searching is same as checking for 'membership'.



The order of elements in a set does not matter, while in a list it does.

Lists

- List searching
 - performed recursively : first find if the desired item is the same as the head of the list, if so then *success*.
 - If it is not, then check if the desired item is in the head of the tail...continue ...if we come to the end of list then *fail*.
 - Implemented using predicate `member (X, L) :` the goal `member` is true if X is in List L

Lists

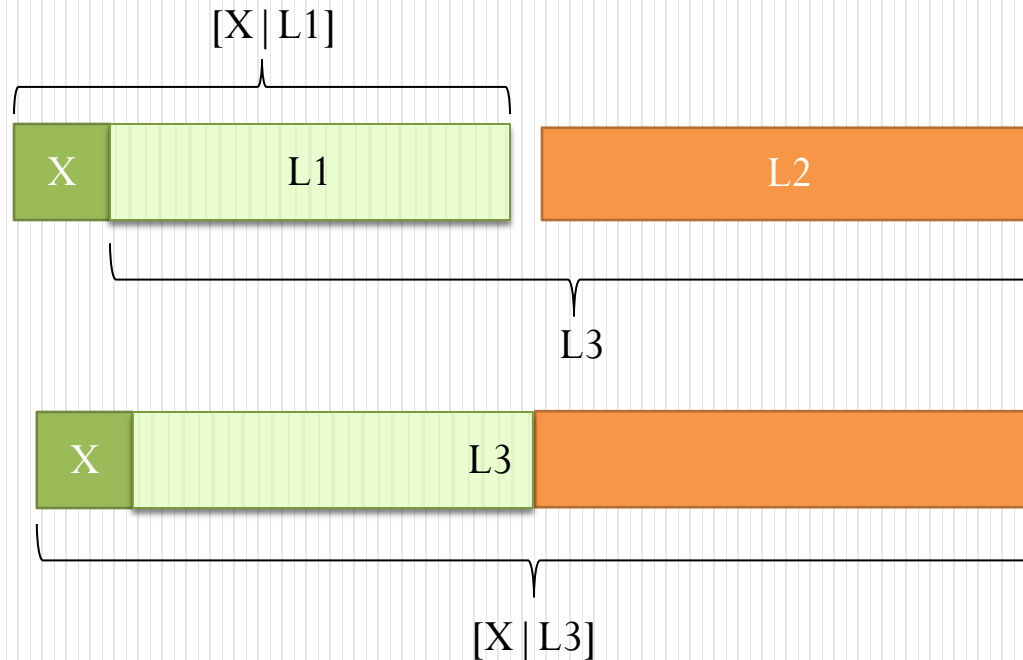
- X is member of L, if either
 - X is the head of L, or
 - X is a member of the tail of L.

Note the anonymous variable '_' used for the tail, we really don't care about the tail in this fact.

```
member(X, [X|_]). /* X is a member of the list that has X
as its head */
member(X, [_|L]) :- member(X, L). /* X is a member of the
list if X is a member of the tail of the list */
```

- Joining Lists
 - append(L1, L2, L3) : L1 and L2 are joined to form L3.
 - If L1 is empty list then L2 and L3 are the same list.
 - If L1 is not empty then it has a head and a tail [X|L1]:

Lists



- First element of the first list (X) will always be the first element of L3.
- L2 is appended to the tail of the first list (L1) to form the tail (L3) of the third list.

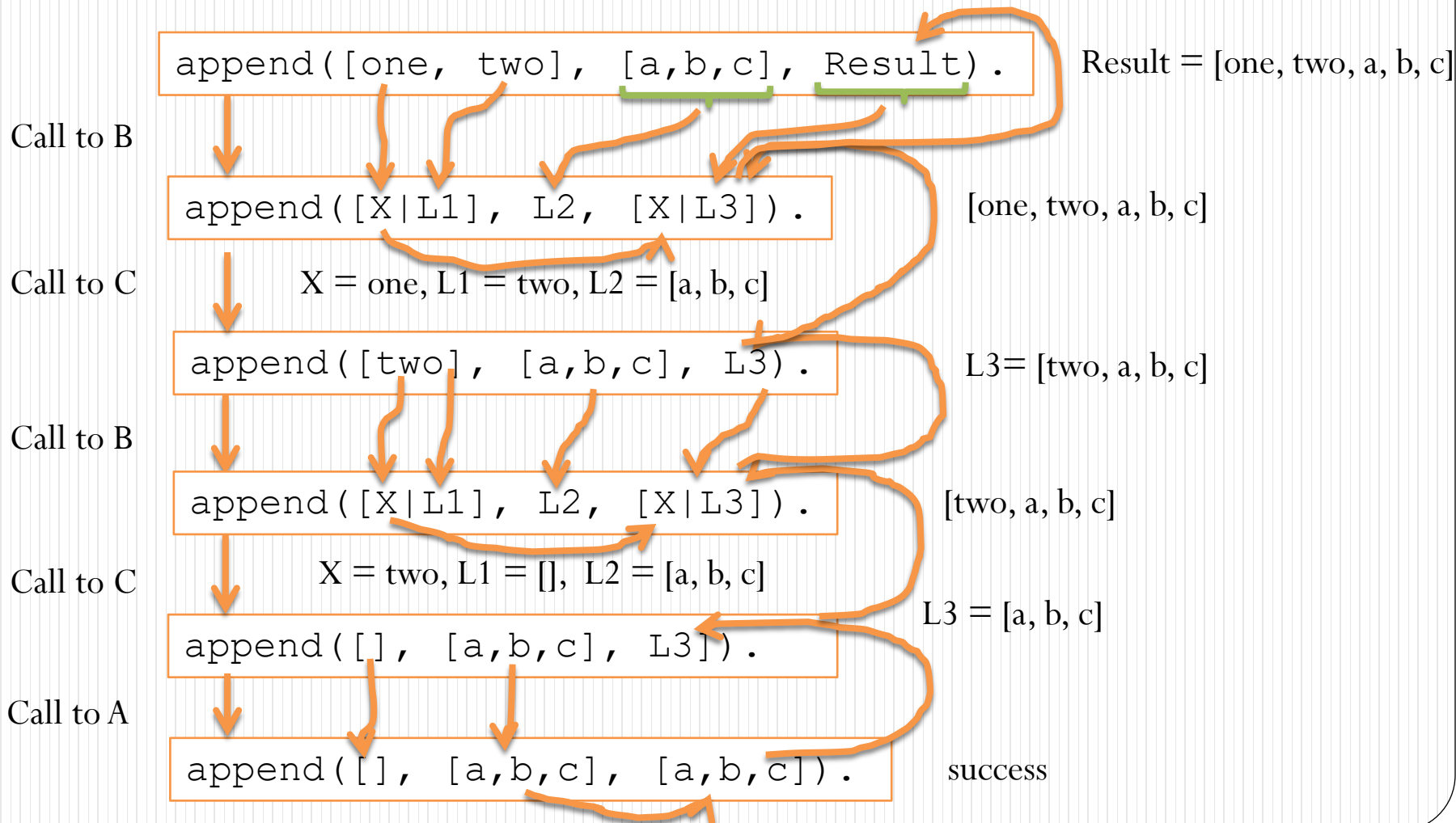
A `append([], L, L).` /* fact 1 */

B `append([X|L1], L2, [X|L3]) :-`

C `append(L1, L2, L3).` /* rule 1 */

`?- append([one, two], [a,b,c], Result).`

`Result = [one, two, a, b, c]`



Arithmetic

- Prolog provides a number of basic arithmetic tools.

Basic Arithmetic Operators

+	addition
-	subtraction
*	multiplication
/	division
mod	modulo

```
?- X = 3+2.  
X = 3+2  
yes  
  
?- X is 3+2.  
X = 5.
```

Note that that $+$, $-$, $/$ and $*$ do not carry out any arithmetic. Expressions such as $3+2$, $4-7$, $5/5$ are ordinary Prolog terms.

- To force Prolog to actually evaluate arithmetic expressions, we have to use **is**

Arithmetic

Comparison Operators

$X > Y$ X is greater than Y
 $X < Y$ X is less than Y
 $X \geq Y$ X is greater than or equal to Y
 $X \leq Y$ X is less than or equal to Y
 $X =: = Y$ the values of X and Y are equal

?- $1+2 =: = 2+1$

yes

? - $1+2 = 2+1.$

no

?- $1+A = B+2.$

$A=2$

$B=1$

$X = Y$ and $X =: = Y$ what is the difference?

$X = Y$

cause matching of the objects.
possibly instantiate some variables in
 X and Y if they match.
no evaluation.

$X =: = Y$

causes arithmetic evaluation.
cannot cause instantiation.

cut (!)

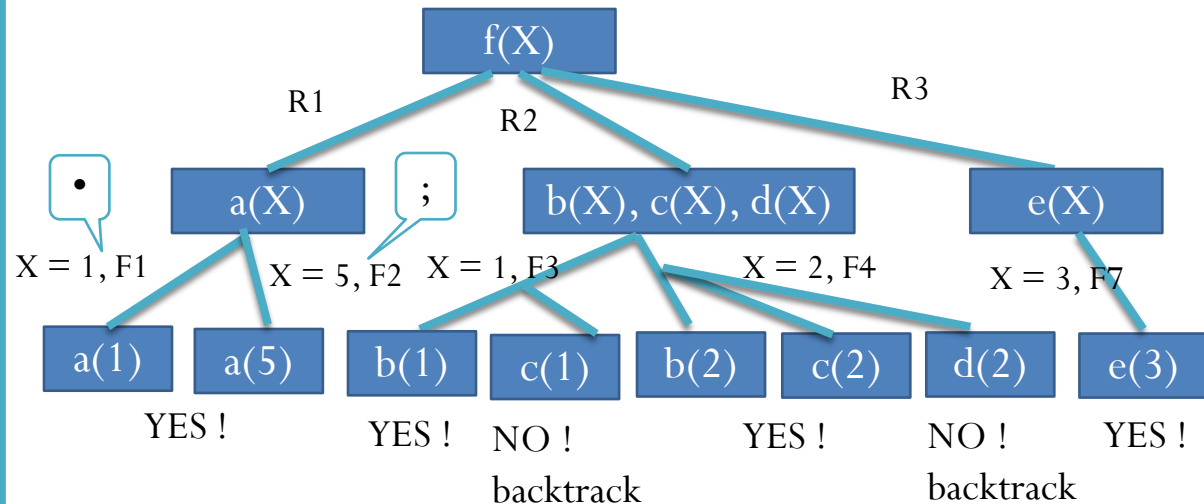
- a special atom used with clauses to control backtracking.
 - Example: $f(X) :- a(X), b(X), !, c(X).$
- goal that always succeeds.
- When a cut is encountered as a goal
 - all choices made since the parent goal (here $f(x)$) was invoked are committed.
 - all other alternatives are discarded.
 - an attempt to re-satisfy any goal between the parent goal and the cut goal will fail.

cut(!)

```
/* without cut */
```

```
f(X) :- a(X). %R1
f(X) :-      %R2
    b(X), c(X), d(X).
f(X) :- e(X). %R3
a(1). %F1
a(5). %F2
b(1). %F3
b(2). %F4
c(2). %F5
d(3). %F6
e(3). %F7
?- f(X). %query
X = 1; %answer
X = 5;
X = 3.
```

Execution Trace

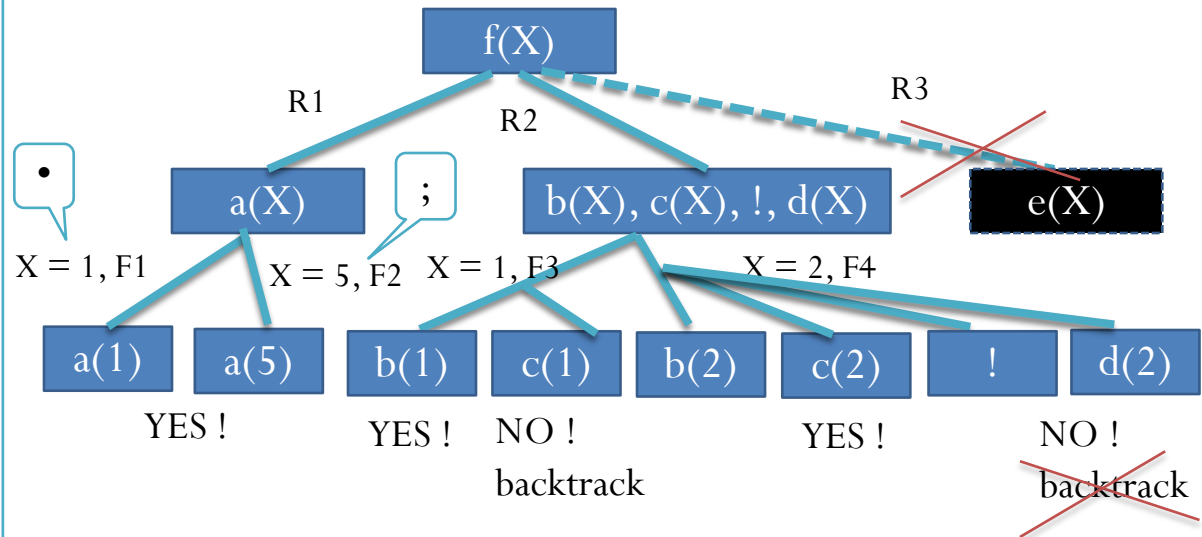


cut(!)

```
/* cut */
```

```
f(X) :- a(X). %R1
f(X) :-      %R2
    b(X), c(X), !, d(X).
f(X) :- e(X). %R3
a(1). %F1
a(5). %F2
b(1). %F3
b(2). %F4
c(2). %F5
d(3). %F6
e(3). %F7
?- f(X). %query
X = 1; %answer
X = 5;
false.
```

Execution Trace



cut(!)

$f \text{ :- } a, b, c, !, p, q, r.$

- once cut is reached it commits
 - this particular clause for f
 - to the choices made when evaluating a, b, c .
- still free to backtrack
 - among p, q, r
 - among the alternatives for choices that were made before reaching the goal f .

cut(!)

`/* without cut */`

```
f(X,Y) :- a(X,Y) .  
f(0,0) .  
a(X,Y) :- b(X), c(Y) .  
b(1) .  
b(2) .  
b(3) .  
c(4) .  
c(5) .
```

```
?- f(X,Y) .
```

```
/* write all the  
values of X and Y */
```

```
?- f(X,Y) .
```

```
X = 1,  
Y = 4 ;
```

```
X = 1,  
Y = 5 ;
```

```
X = 2,  
Y = 4 ;
```

```
X = 2,  
Y = 5 ;
```

```
X = 3,  
Y = 4 ;
```

```
X = 3,  
Y = 5 ;
```

```
X = Y, Y = 0 .
```


cut(!)

```
/* with cut */
```

```
f(X,Y) :- a(X,Y) .  
f(0,0) .  
a(X,Y) :- b(X),!, c(Y) .  
b(1) .  
b(2) .  
b(3) .  
c(4) .  
c(5) .  
  
?- f(X,Y) .  
  
/* write all the values  
of X and Y */
```

```
?-f(X,Y) .
```

```
X = 1,
```

```
Y = 4 ;
```

```
X = 1,
```

```
Y = 5 ;
```

```
X = Y, Y = 0.
```

Using cut (!)

- cut used for confirming the choice of a rule.
- Example: consider a predicate `max` that succeeds if the third argument is the maximum of the first two.

```
?- max(2, 5, 5) .  
yes  
?-max(5, 2, 5) .  
yes  
?-max(5, 5, 5) .  
yes  
?-max(2, 5, 7) .  
no  
?-max(2, 5, X) .  
X = 5
```

```
max(X, Y, Y) :- X <= Y.  
max(X, Y, X) :- X > Y.
```

What is the problem?

```
?-max(2, 3, X) .  
X = 3;  
no.
```

When asked for more solutions, it will try to satisfy second clause which is waste of time.

Using cut (!)

We need to tell Prolog that on no account the second rule ever to be tried if $X \leq Y$.

One way of doing this is to use cut.

```
max(X, Y, Y) :- X <= Y, !.  
max(X, Y, X) :- X > Y.
```

- How this works:
 - If $X \leq Y$ succeeds then the cut commits to this choice, and the second clause of max is not considered.
 - If $X \leq Y$ fails, then Prolog goes on to the second clause.

```
?-max(2, 3, X).  
X = 3.
```

Note that this cut does not change the meaning of the program and gives exactly the same answer as the previous one.

Using cut (!)

- A change in the order of clauses may affect the declarative meaning of programs with cut.
- Cuts that change the declarative meaning: **Red cuts** and those that do not change : **Green cuts**.

```
f :- a, b.  
f :- c.
```

Declarative meaning : p is true if and only if a and b are true or c is true.

$$P \Leftrightarrow (a \ \& \ b) \vee c$$

Change the order of the clauses and the meaning remains same

```
/* insert cut */  
f :- a,!,b.  
f :- c.
```

$$P \Leftrightarrow (a \ \& \ b) \vee (\sim a \ \& \ c)$$

```
/* insert cut */  
/* swap clauses */  
f :- c.  
f :- a,!,b.
```

$$P \Leftrightarrow c \vee (a \ \& \ b)$$

Negation as failure

- `fail` :
 - special built-in predicate that will immediately fail when Prolog encounters it as goal.
 - may not sound useful – but remember when prolog fails, it tries to backtrack.
 - can be viewed as an instruction to force backtracking.
- `cut-fail` combination allows to define exceptions to general rules (`fail` forces backtracking and `cut` blocks backtracking).

Negation as failure

- Example: cut-fail combination
 - Sam likes chocolates **except** the white ones.

```
/* Sam likes chocolates */  
likes(sam, X):- chocolate(X).
```

```
/* If X is a white chocolate then 'Sam likes X' is not  
true otherwise if X is a chocolate then Sam likes it */  
  
likes(sam, X):- white_chocolate(X),!,fail.  
likes(sam,X) :- chocolates(X).
```

- The first rule takes care of the white chocolates: If X is a white chocolate then the cut will prevent backtracking (thus blocking access to the second rule) and fail will cause the failure.



Negation as failure

- The cut-fail combination allows to define some form of negation called **negation as failure**, defined as:

```
neg(Goal) :- Goal, !, fail.  
neg(Goal) .
```

- If `Goal` succeeds then `neg(Goal)` fails, otherwise `neg(Goal)` succeeds.
- Using `neg(Goal)`, the Sam and chocolate example can be rewritten as:

```
likes(Sam, X) :- chocolate(X), neg(white_chocolate(X)).
```

Negation as failure

- In standard Prolog the operator `\+` means negation as failure.



Negation as failure does not exactly correspond to negation in mathematical logic.

```
likes(sam, X):- chocolate(X),
                 \+ white_chocolate(X).
chocolate(X) :-
    white_chocolate(X).
chocolate(X):-
    milk_chocolate(X).
chocolate(X):-
    dark_chocolate(X).
milk_chocolate(dairymilk).
white_chocolate(milkybar).
dark_chocolate(bournville).
```

```
?- likes(sam,X).
X = dairymilk;
X = bournville;
no
```

Version 1

```
likes(sam, X):-
    \+ white_chocolate(X),
    chocolate(X).
chocolate(X) :-
    white_chocolate(X).
chocolate(X):-
    milk_chocolate(X).
chocolate(X):-
    dark_chocolate(X).
milk_chocolate(dairymilk).
white_chocolate(milkybar).
dark_chocolate(bournville).
```

```
?- likes(sam,X).
no
```

Version 2

Negation as failure

What is the problem here?

```
likes(sam, X) :-  
    \+ white_chocolate(X),  
    chocolate(X).  
chocolate(X) :-  
    white_chocolate(X).  
chocolate(X) :-  
    milk_chocolate(X).  
chocolate(X) :-  
    dark_chocolate(X).  
milk_chocolate(dairymilk).  
white_chocolate(milkybar).  
dark_chocolate(bournville).  
  
?- likes(sam, X).  
no
```

Logically 'chocolate(X) and not white_chocolate(X)' is equivalent to 'not white_chocolate(X) and chocolate(X) but ... in Prolog the program behaves differently when the order of the goals is changed.

First goal is `\+ white_chocolate(X)` since `white_chocolate(X)` unifies with second fact (`X = milkybar`) so it succeeds. Hence, `\+ white_chocolate(X)` fails and the original query too.

In the version 1 `\+` is used only after `X` is instantiated and in this version `\+` is used before instantiation.

Summary

- In this tutorial we learnt:
 - lists: an important data structure often used in Prolog.
 - basic operations on list: finding membership, joining using recursion.
 - basic arithmetic in Prolog.
 - cut and fail built-in predicates to control backtrack.
 - problems with cut and fail.